

Rebel Profiler - Frontend Handoff Brief (v1.4.0)

***Ye document ek AI/developer ko diya ja sakta hai** - isme project ka poora context, live API contract, aur frontend ka complete plan hai. Padhte hi kaam shuru ho sakta hai, bina codebase 100% padhe.*

1. Project ek paragraph mein

Rebel Profiler ek **local-first, case-based security recon framework** hai jiska front door Hermes hai - ek agentic chat interface jo plain language samajhta hai. Sab kuch ek laptop par chalta hai: LLM (Qwen2.5-Coder-7B GGUF, llama.cpp) local hai, koi data bahar nahi jata (RP_LLM__ENGINE=external opt-in hi hai). Har action 6 gates se guzarta hai (scope -> policy -> risk -> confirmation -> approval -> evidence) aur har claim hash-chained evidence se backed hota hai. MIT licensed, stdlib-only core, Python 3.11+.

User flow: case banao -> scope authorize karo -> case activate -> tools/Hermes se data collect karo -> claims + evidence auto-build hote hain -> surface graph + report nikalte hain -> sab tamper-evident.

2. Jo already kaam karta hai (verified live)

- * **CLI** (``rebel-profiler ...``): 30+ subcommands - case, scope, intel, report, surface, audit, evidence, agent, hermes, llm, browser, serve, ops...
- * **Hermes chat** (``hermes` ya `rp talk``): plain language -> LLM khud tools chalata hai (compact tool-contract prompt, stop sequences, bare-JSON parser)
 - * ``tools/rp``: single wrapper - ``rp`` = status, ``rp talk`` = chat, ``rp "<goal>"`` = ek prompt
- * **Browser bridge** (port 8765, token-gated): LLM operator ke browser ko use karta hai
- * **Read-only API** (``serve``): frontend ke liye - neeche contract
- * **Tests**: 776 passing (Python 3.11/3.12, GitHub Actions green)

3. Live API contract (frontend isi pe banega)

``rebel-profiler serve <case-id> --port 8899 --token <secret>``

Read-only, GET-only, token-gated. Auth: token ka SHA-256 hex banta hai; header mein wahi hex bhejo:

```
Authorization: Bearer <sha256_hex_of_secret>
```

Endpoint | Kya deta hai

GET / | banner: {service, version, case_id}

GET /healthz | {ok: true}

GET /state | {case_id, claims, evidence, audit_head} - counts + audit chain head hash

GET /events | {events: [...]} - recent 50 structured events

Errors: 401 unauthorized (galat/missing token), 503 API disabled (token hi set nahi), 404 (unknown path), 405 (GET ke siwa kuch nahi).

Verified live example (/state):

```
{"audit_head": "51eeaf6f...", "case_id": "394c8b4dd5a1", "claims": 6, "evidence": 4}
```

Browser bridge (``rp bridge``, port 8765)

GET /healthz -> {ok, case_id, scope:[...]} - GET /tasks -> {tasks:[...]}
- POST /ack, POST /result (extension in kaam). Bearer token: /tmp/rp_bridge_token.

Baaki data (v1 frontend CLI se lega - ye already JSON dete hain)

```
rebel-profiler case list -o json
rebel-profiler report <case-id> -o json
rebel-profiler surface map <case-id> -o json      # nodes + edges graph
rebel-profiler intel claims <case-id> -o json
rebel-profiler intel collect <case> <action> <target> -o json
rebel-profiler hermes ... -o json                # poora transcript + calls
```

4. Frontend plan (phase-wise)

Stack suggestion: koi bhi modern stack chalega - Vite + React/Svelte ya sirf server-rendered + htmx. Core wala stdlib-only promise frontend pe apply nahi hota (frontend alag package/folder hoga, e.g. frontend/).

Phase F1 - Read-only dashboard (sabse pehle)

- * Case list + case detail (claims table, evidence chain status, audit head)
- * `state` + `events` ko poll karke live counters
- * Report view: findings, confidence, evidence links
- * Surface graph render (nodes/edges `surface map` JSON se - force-directed ya simple SVG)

Phase F2 - Case management (write path, CLI wrapper se)

- * New case form -> `case create` + `scope add` + `case activate`
- * Collect buttons: adapters list (`adapters -o json`) -> form -> `intel collect`
- * Sab writes **backend proxy** ke through (browser se direct CLI nahi)

Phase F3 - Hermes chat UI

- * WebSocket/SSE proxy -> `hermes` process ki stdout stream
- * Tool-call timeline dikhana: `[t1] ok claims\_list`, final answer highlight
- * First-turn heads-up UI (CPU prefill ~2.5 min) + turn/seconds footer

Phase F4 - Polish

- * Bridge status widget (`/healthz` on 8765), token paste popup
- * Audit chain visual (hash links), evidence diff viewer
- * Dark Kali-style theme, Urdu/Roman-Urdu labels option

5. Design rules (inhein todkar mat chalna)

- * **Local-first**: frontend bhi local serve hoga; koi cloud call nahi
- * **Fail-closed**: scope/policy UI se bypass nahi hoga - writes broker ke gates se hi
- * **Evidence law**: UI mein har finding ka evidence link dikhna chahiye
- * **JSON-first**: backend se hamesha `-o json` lete hain, human text parse kabhi nahi
- * **Ekdum honest**: slow turn hai to "working..." + expected time dikhao, fake progress nahi

6. Dev quickstart

```
cd "/home/rebel/freebuff/rebel profiler"
.venv/bin/python -m pytest tests/      # 776 passing
./tools/rp                             # status panel
```

```
rebel-profiler serve <case-id> --port 8899 --token <secret> # API  
curl -H "Authorization: Bearer <sha256(secret)>" localhost:8899/state
```

Frontend folder: frontend/ (abhi nahi hai - tum banana). Is brief ko padhkar seedha Phase F1 shuru kar sakte ho.

*Generated live on hardware: case 394c8b4dd5a1, target www.hplovecraft.com
(6 claims, 4 evidence, audit chain OK), hermes 3-turn agentic run verified.*